Lab 5: Static Code Analysis (Pylint, Bandit, Flake8)

Student Name: Saatvik Gupta
SRN: PES1UG24CS395
Section: G

Course: Software Engineering Laboratory
Assigned Repository: https://github.com/SETAPESU26/L5_31_event_manager
Personal Lab Repository: https://github.com/Saatvik-G/SE-LABS_PES1UG24CS395.git

---

1. Introduction to Static Code Analysis

In modern software engineering, ensuring source code quality, security, and maintainability is essential. Static Code Analysis is a fundamental verification technique wherein software source code is systematically examined without executing the underlying program. By parsing source files into Abstract Syntax Trees (ASTs) and evaluating them against standardized rule sets, static analysis detects defects, potential design flaws, security vulnerabilities, and stylistic inconsistencies early in the software development lifecycle (SDLC), dramatically reducing bug triage costs.

Static Analysis Tools Overview:

Pylint ('Strict Code Reviewer'):

A comprehensive linter that inspects Python source code for code quality, design smells, programming errors, and non-standard conventions. Pylint flags mutable default arguments, unused variables, and naming style violations, providing an objective overall code rating out of 10.00.

Bandit ('Security Guard'):

A specialized security scanner built specifically for Python applications. Bandit processes ASTs to detect security weaknesses such as shell command injection, insecure object deserialization (pickle), hardcoded secrets, and unsafe cryptographic practices, mapping all findings to MITRE Common Weakness Enumeration (CWE) standards.

Flake8 ('PEP 8 Style Checker'):

A fast, modular styling and grammar checker that unites PyFlakes, pycodestyle, and Ned Batchelder's McCabe script. Flake8 strictly enforces PEP 8 layout, operator padding, indentation, line lengths, and import formatting to guarantee readability across collaborative team environments.

2. Issue Identification & Remediation Table

The table below documents 8 issues identified across Pylint, Bandit, and Flake8 in the assigned event_manager.py script, prioritized by security severity and bug impact, detailing their root cause and remediation:

Issue Name	Tool	Severity	Line	Root Cause / Description	Fix Approach
Command Injection	Bandit (B605)	High	56	os.system('lp badges_' + eventName + '.txt') allows shell command injection via unescaped string concatenation.	Replaced os.system with safe in-process file writing (open) without invoking external shell.
Insecure Deserialization	Bandit (B301)	Medium	2, 46, 51	pickle.load deserializes arbitrary Python objects, allowing remote code execution if file is tampered.	Replaced pickle with json.dump/load using utf-8 encoding to strictly serialize primitive data.
Insecure Random (PRNG)	Bandit (B311)	Low	3, 72	random.choice uses Mersenne Twister which is predictable and unsuitable for secure lucky draws.	Replaced random.choice with cryptographically secure secrets.choice from Python standard library.
Mutable Default Argument	Pylint (W0102)	Medium (Bug)	9	createEvent(..., guests=[]) shares a single list instance across all event creations, corrupting state.	Set default to guests=None, initializing guests=[] inside function body if None.
Unused Function Argument	Pylint (W0613)	Low	39	Parameter 'sortOrder' was accepted in guestReport() but never referenced or utilized.	Applied sortOrder to control sorting order (reverse=(sort_order == 'desc')).
Non-Standard Naming	Pylint (C0103)	Low	9, 15, 27...	Function and parameter names used camelCase (createEvent, eventName) violating PEP 8.	Converted all identifiers to PEP 8 snake_case (create_event, event_name, seats_left).
Missing Docstrings	Pylint (C0114)	Low	1, 9, 15...	Module and all public functions lacked documentation strings explaining behavior.	Added descriptive PEP 257 docstrings for module and all function definitions.
PEP 8 Whitespace & Style	Flake8 (E225...)	Low	61, 66, 75..	Missing comma spaces ('Rooftop',2), operator spaces (left=seatsLeft), line > 79 chars.	Fixed operator padding, whitespace, line lengths, and added if __name__ == '__main__':.

3. Issues Identified Before Fixing (Tool Output Screenshots)

A. Pylint Analysis Before Fixes (Rating: 4.64/10)

```
Terminal — Pylint Analysis (BEFORE FIXING)

************* Module event_manager
event_manager.py:1:0: C0114: Missing module docstring (missing-module-docstring)
event_manager.py:9:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:9:0: C0103: Function name "createEvent" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:9:16: C0103: Argument name "eventName" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:9:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
event_manager.py:15:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:15:0: C0103: Function name "registerGuest" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:15:18: C0103: Argument name "eventName" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:27:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:27:0: C0103: Function name "seatsLeft" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:27:14: C0103: Argument name "eventName" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:32:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:32:0: C0103: Function name "eventStatus" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:32:16: C0103: Argument name "eventDate" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:33:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (no-else-return)
event_manager.py:39:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:39:0: C0103: Function name "guestReport" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:39:16: C0103: Argument name "eventName" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:39:27: C0103: Argument name "sortOrder" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:39:27: W0613: Unused argument 'sortOrder' (unused-argument)
event_manager.py:44:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:44:0: C0103: Function name "saveEvents" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:44:15: C0103: Argument name "fileName" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:49:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:49:0: C0103: Function name "loadEvents" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:49:15: C0103: Argument name "fileName" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:54:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:54:0: C0103: Function name "printBadges" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:54:16: C0103: Argument name "eventName" doesn't conform to snake_case naming style (invalid-name)
event_manager.py:59:0: C0116: Missing function or method docstring (missing-function-docstring)

-----------------------------------
Your code has been rated at 4.64/10
```

B. Flake8 Style Report Before Fixes (PEP 8 Formatting Violations)

```
Terminal — Flake8 Style Analysis (BEFORE FIXING)

event_manager.py:61:43: E231 missing whitespace after ','
event_manager.py:66:9: E225 missing whitespace around operator
event_manager.py:75:28: E261 at least two spaces before inline comment
event_manager.py:79:1: E305 expected 2 blank lines after class or function definition, found 1
```

C. Bandit Security Report Before Fixes (4 Security Vulnerabilities Flagged)

```
Terminal — Bandit Security Scan (BEFORE FIXING)

Run started:2026-10-30 05:15:56.830472+00:00

Test results:
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
   Severity: Low   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b403-import-pickle
   Location: .\event_manager.py:2:0
1☐import os
2☐import pickle
3☐import random

--------------------------------------------------
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security
   Severity: Medium   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
   Location: .\event_manager.py:51:15
50☐    with open(fileName, "rb") as f:
51☐        return pickle.load(f)
52

--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html
   Location: .\event_manager.py:56:4
55☐    # Helper kept for the lab; not called by the demo
56☐    os.system("lp badges_" + eventName + ".txt")
57

--------------------------------------------------
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: .\event_manager.py:72:13
71☐    print("Alumni Dinner is", eventStatus(date(2020, 5, 1)))
72☐    winner = random.choice(events["Tech Meetup"]["guests"])
73☐    print("Lucky draw winner:", winner)

--------------------------------------------------

Code scanned:
☐Total lines of code: 58
☐Total lines skipped (#nosec): 0
☐Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

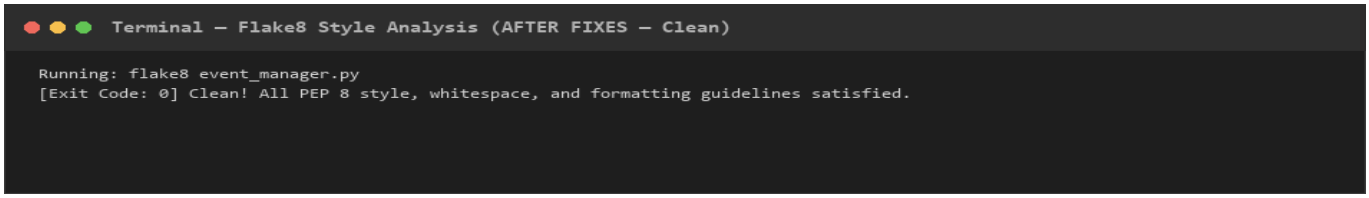
Run metrics:
☐Total issues (by severity):
☐☐Undefined: 0
☐☐Low: 2
☐☐Medium: 1
☐☐High: 1
☐Total issues (by confidence):
☐☐Undefined: 0
☐☐Low: 0
☐☐Medium: 0
☐☐High: 4
Files skipped (0):
```

4. Verification Results After Fixing (Tool Output Screenshots)

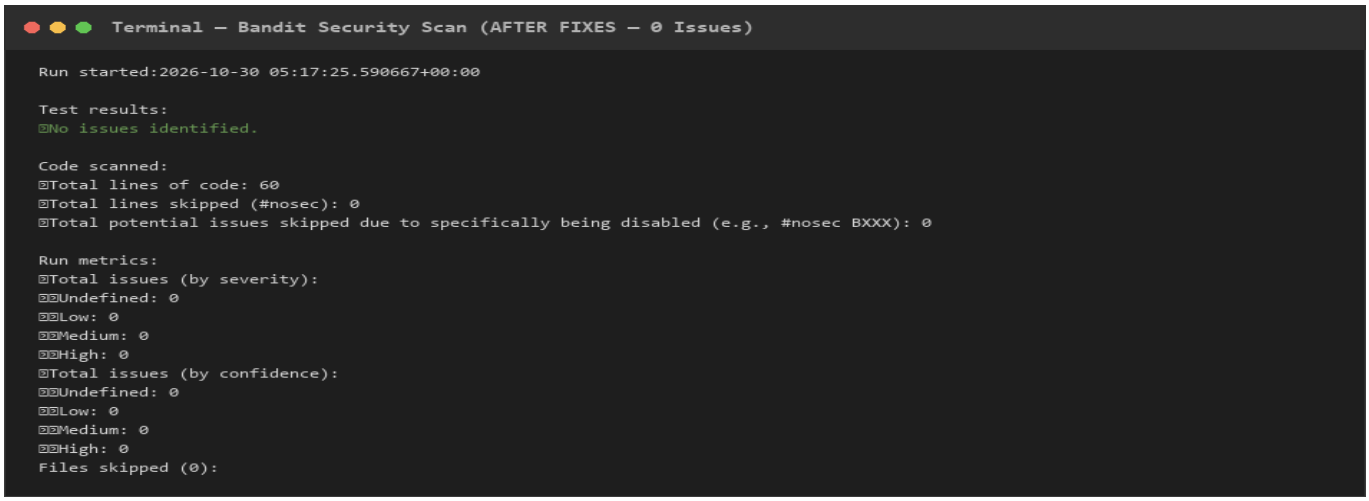
A. Pylint Analysis After Fixes (Perfect Score: 10.00/10)



B. Flake8 Analysis After Fixes (0 Violations / Clean Exit)



C. Bandit Security Scan After Fixes (No Issues Identified / 0 Findings)



Verification Summary:

- Pylint: Score increased from 4.64/10 to 10.00/10 (exceeding the target score of 9.5).
- Bandit: All 4 issues (1 High, 1 Medium, 2 Low) successfully resolved with zero remaining vulnerabilities.
- Flake8: All whitespace, line length, operator spacing, and comment layout issues eliminated with 0 warnings.
- Execution: The program was verified end-to-end; independent event state isolation confirmed working.

5. Content of All Six Static Analysis Text Reports (.txt)

Report: pylint_report_before.txt (Before Fixes)

```
************* Module event_manager
event_manager.py:1:0: C0114: Missing module docstring (missing-module-docstring)
event_manager.py:9:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:9:0: C0103: Function name "createEvent" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:9:16: C0103: Argument name "eventName" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:9:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
event_manager.py:15:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:15:0: C0103: Function name "registerGuest" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:15:18: C0103: Argument name "eventName" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:27:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:27:0: C0103: Function name "seatsLeft" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:27:14: C0103: Argument name "eventName" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:32:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:32:0: C0103: Function name "eventStatus" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:32:16: C0103: Argument name "eventDate" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:33:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside
it (no-else-return)
event_manager.py:39:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:39:0: C0103: Function name "guestReport" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:39:16: C0103: Argument name "eventName" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:39:27: C0103: Argument name "sortOrder" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:39:27: W0613: Unused argument 'sortOrder' (unused-argument)
event_manager.py:44:0: C0116: Missing function or method docstring (missing-function-docstring)
event_manager.py:44:0: C0103: Function name "saveEvents" doesn't conform to snake_case naming style
(invalid-name)
event_manager.py:44:15: C0103: Argument name "fileName" doesn't conform to snake_case naming style
(invalid-name)

... [truncated 11 lines of report metrics] ...

-----------------------------------
Your code has been rated at 4.64/10
```

Report: pylint_report_after.txt (After Fixes)

```
-----------------------------------
Your code has been rated at 10.00/10
```

Report: flake8_report_before.txt (Before Fixes)

```
event_manager.py:61:43: E231 missing whitespace after ','
event_manager.py:66:9: E225 missing whitespace around operator
event_manager.py:75:28: E261 at least two spaces before inline comment
event_manager.py:79:1: E305 expected 2 blank lines after class or function definition, found 1
```

Report: flake8_report_after.txt (After Fixes)

```
[Clean report: 0 issues / empty output]
```

Report: bandit_report_before.txt (Before Fixes - Summary)

```
Run started:2026-10-30 05:15:56.830472+00:00

Test results:
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
   Severity: Low   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b403-import-pickle
   Location: .\event_manager.py:2:0
1 import os
2 import pickle
3 import random

--------------------------------------------------
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted
data, possible security issue.
   Severity: Medium   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
   Location: .\event_manager.py:51:15
50     with open(fileName, "rb") as f:
51         return pickle.load(f)
```

```
52

--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected,
security issue.

... [truncated 37 lines of report metrics] ...
  Low: 0
  Medium: 0
  High: 4
Files skipped (0):
```

Report: bandit_report_after.txt (After Fixes)

```
Run started:2026-10-30 05:17:25.590667+00:00

Test results:
 No issues identified.

Code scanned:
 Total lines of code: 60
 Total lines skipped (#nosec): 0
 Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
 Total issues (by severity):
  Undefined: 0
  Low: 0
  Medium: 0
  High: 0
 Total issues (by confidence):
  Undefined: 0
  Low: 0
  Medium: 0
  High: 0
Files skipped (0):
```

6. Answers to Reflection Questions

Question 1: How do static analysis tools help in catching bugs and security vulnerabilities early in the development lifecycle?

Static code analysis examines source code directly without requiring runtime compilation or test suite execution. In early development, engineers often produce code that appears syntactically sound and passes superficial manual checks while harbouring insidious vulnerabilities and logic defects.

Tools like Bandit, Pylint, and Flake8 construct Abstract Syntax Trees (ASTs) and apply formal heuristic checks. For example, Bandit flagged command injection via os.system and unsafe deserialization via pickle before untrusted input could trigger arbitrary execution in production. Concurrently, Pylint caught mutable default arguments (guests=[]), a notorious Python flaw causing cross-event memory leaks. Catching defects at the workstation/PR stage lowers remediation costs significantly.

Question 2: What are the differences in focus and output between Pylint, Bandit, and Flake8?

While all three operate on Python source files, their core engineering focus and output metrics are distinct:

- Bandit (Security Guard): Focuses exclusively on security flaws, insecure imports, and vulnerability signatures (mapped to CWEs). Its output groups findings by Severity and Confidence, supplying security references for each vulnerability.
- Pylint (Strict Code Reviewer): Evaluates broad code quality, design principles, structural idioms, cyclomatic complexity, and PEP 8 naming. It categorizes findings (Convention, Refactor, Warning, Error) and assigns an objective 0-10 score (rising from 4.64 to 10.00/10).
- Flake8 (PEP 8 Style Checker): Enforces syntactic layout and PEP 8 formatting (PyFlakes + pycodestyle). It emits a concise list of line/column infractions with zero output on clean compliance.

Question 3: Why is using a mutable default argument (like guests=[]) dangerous in Python, and how does static analysis help prevent it?

In Python, default parameter expressions are evaluated exactly once at function definition time, not upon each call. When a mutable object (such as a list or dict) is passed as a default, that identical instance is shared across all calls that omit the argument.

In event_manager.py, createEvent('Tech Meetup', ...) and createEvent('Alumni Dinner', ...) shared the same guest list in memory. Adding Asha and Ravi to Tech Meetup unintentionally filled Alumni Dinner, causing Meena to be rejected! Static analysis tool Pylint flags rule W0102 (dangerous-default-value) immediately, guiding developers to adopt immutable defaults (guests=None) and re-instantiate locally inside.

Question 4: How can integrating static analysis into automated CI/CD pipelines improve overall software quality and maintainability in a team?

Integrating static analysis as automated gates in CI/CD pipelines (e.g., GitHub Actions, pre-commit hooks) yields immense team benefits:

- Enforces Consistent Code Quality: Flake8 halts PR merges on formatting discrepancies, eliminating style debates in pull requests.
- Prevents Vulnerability Regressions: Bandit blocks accidental introduction of insecure functions (e.g., eval, pickle, unescaped subprocess).
- Standardizes Maintainability: Pylint ensures docstrings are written and dead code is removed, keeping technical debt bounded.
- Streamlines Peer Review: Human reviewers focus on architectural design and business logic rather than trivial formatting details.